

D-Models and E-Models: Diversity-Stability Trade-offs in the Sampling Behavior of Large Language Models

Jia Gu

gujia24s@ict.ac.cn

State Key Laboratory of AI Safety

Institute of Computing Technology, Chinese Academy of Sciences

University of Chinese Academy of Sciences
Beijing, China

Huawei Shen

shenhuawei@ict.ac.cn

State Key Laboratory of AI Safety

Institute of Computing Technology, Chinese Academy of Sciences

University of Chinese Academy of Sciences
Beijing, China

Liang Pang*

pangliang@ict.ac.cn

State Key Laboratory of AI Safety

Institute of Computing Technology, Chinese Academy of Sciences

University of Chinese Academy of Sciences
Beijing, China

Xueqi Cheng

cxq@ict.ac.cn

State Key Laboratory of AI Safety

Institute of Computing Technology, Chinese Academy of Sciences

University of Chinese Academy of Sciences
Beijing, China

Abstract

The predictive probability of the next token (P_{token}) in large language models (LLMs) is inextricably linked to the probability of relevance for the next piece of information, the purchase probability of the next product, and the execution probability of the next action—all of which fall under the scope of the task-level target distribution (P_{task}). While LLMs are known to generate samples that approximate real-world distributions, whether their fine-grained sampling probabilities faithfully align with task requirements remains an open question. Through controlled distribution-sampling simulations, we uncover a striking dichotomy in LLM behavior, distinguishing two model types: D-models (e.g. Qwen-2.5), whose P_{token} exhibits large step-to-step variability and poor alignment with P_{task} ; and E-models (e.g. Mistral-Small), whose P_{token} is more stable and better aligned with P_{task} . We further evaluate these two model types in downstream tasks such as code generation and recommendation, revealing systematic trade-offs between diversity and stability that shape task outcomes. Finally, we analyze the internal properties of both model families to probe their underlying mechanisms. These findings offer foundational insights into the probabilistic sampling behavior of LLMs and provide practical guidance on when to favor D- versus E-models. For web-scale applications—including recommendation, search, and conversational agents—our results inform model selection and configuration to balance diversity with reliability under real-world uncertainty.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792636>

CCS Concepts

• **Computing methodologies** → **Natural language generation**; **Probabilistic reasoning**; • **Information systems** → **Web searching and information discovery**.

Keywords

Large Language Models, Probability Distribution Sampling, Large Language Models Interpretation.

ACM Reference Format:

Jia Gu, Liang Pang*, Huawei Shen, and Xueqi Cheng. 2026. D-Models and E-Models: Diversity-Stability Trade-offs in the Sampling Behavior of Large Language Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792636>

1 Introduction

The internet is increasingly relying on large language models (LLMs) to assist in code generation [15, 19] or moderate the exposure of information in search results, news feeds and recommendations [6, 28]. This moderation is not just a matter of convenience, but also a responsibility regarding the transparency of technology and algorithms, and user trust. When these models decide which content to display and how frequently, they are essentially sampling from a distribution of potential items, thereby shaping what users see, believe and do. Therefore, ensuring that this sampling process is auditable and explainable is a crucial aspect of the responsible development of internet technology.

The core foundation of LLMs' capabilities lies in their ability to model and learn probability distributions from large-scale text data. Given a text sequence (context), the model predicts a probability distribution over the entire vocabulary for the next token, denoted as $P(\text{token} \mid \text{context})$. This distribution quantifies the likelihood of each possible next token, and text generation is ultimately performed by sampling from it, which we refer to as P_{token} .

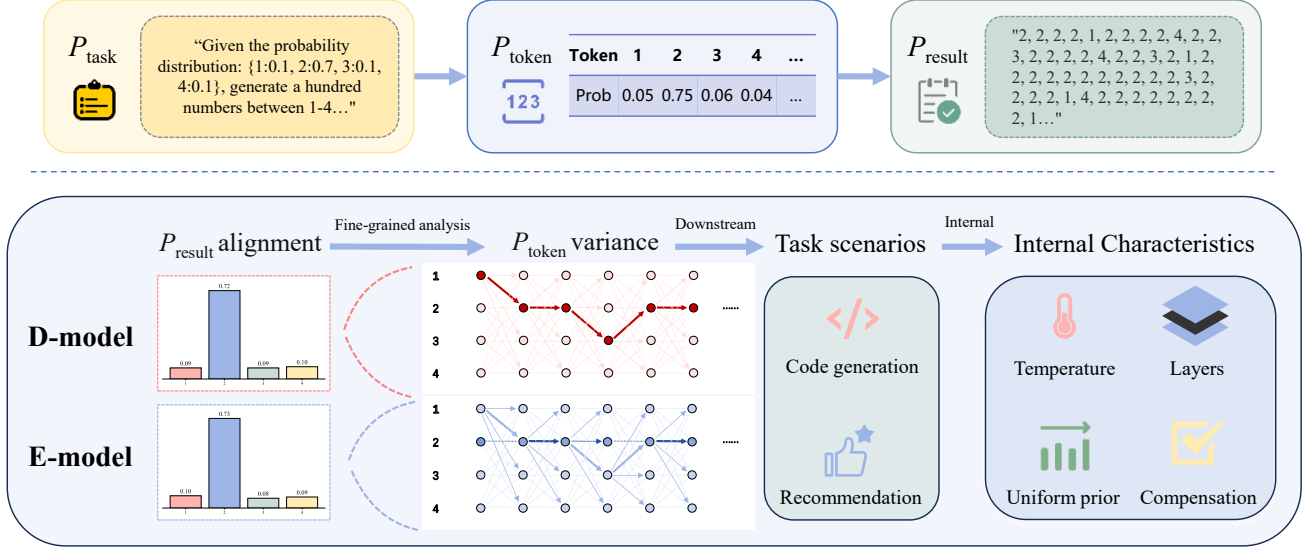


Figure 1: We analyze the probabilistic distribution sampling capabilities of LLMs by combining P_{task} , P_{token} , and P_{result} , and have verified the existence of both the D-model and the E-model. Although there are minor discrepancies in the P_{result} , which aligns somewhat with the P_{task} , they demonstrate distinct patterns at the fine-grained P_{token} level. We have also analyzed both models within downstream task scenarios and their internal characteristics.

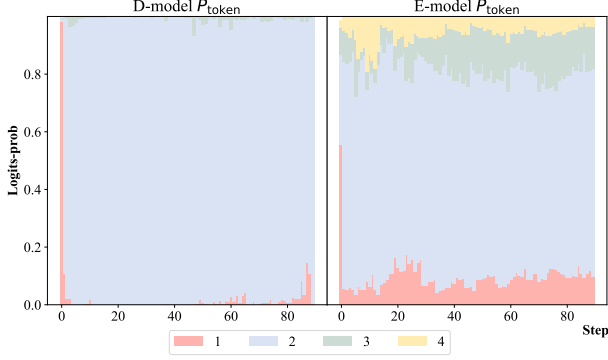


Figure 2: Comparison of P_{token} between D- and E-model for distribution $\{1: 0.1, 2: 0.7, 3: 0.1, 4: 0.1\}$. The horizontal axis denotes the generation step, while the vertical axis represents the corresponding token probability distribution.

Meanwhile, web tasks are inherently uncertain: In search, news feeds, and recommendations, a given user or query context induces a probability distribution over candidate items. When systems decide what to display and how frequently, they effectively sample from this distribution rather than selecting a single optimal item. Recommenders, for instance, aim not only to return the most probable items but also to balance diversity and uncertainty to meet heterogeneous user needs. We therefore model the task objective as a probability distribution over the candidate set, denoted P_{task} .

which captures the intrinsic complexity of the web and provides a principled basis for reasoning under uncertainty.

This raises a critical question: **Can LLMs effectively understand and adapt to different task objective distributions?** In other words, can LLMs generate outputs aligned with the task-level objective distribution P_{task} from their internal token-level probability distribution P_{token} ? This question concerns not only the theoretical foundation of the model's generative capabilities but also its practical performance in complex tasks.

To explore this issue, this paper investigates the generative capabilities of LLMs from a probabilistic perspective. Prior work has examined their probability sampling behavior, including random number generation [13, 26, 29] and coin flipping [11, 17, 31], revealing notable deficiencies [10, 13, 20, 31]. However, these studies are largely empirical and scenario-specific, without systematically examining the mapping between the internal token-level distribution P_{token} and the task-level objective distribution P_{task} , nor their practical implications in real-world tasks.

As shown in Figure 1, this study leverages log probabilities to connect the token-level distribution P_{token} with the task-level target distribution P_{task} and the resulting sampling distribution P_{result} , enabling a joint analysis of LLMs' probability sampling behavior. We observe that P_{token} does not always exhibit extreme confidence, but instead shows two representative patterns: (1) deterministic global planning, where a single token dominates the distribution yet may deviate from P_{task} , and (2) exploratory local planning, characterized by a relatively uniform distribution that better aligns with P_{task} . Accordingly, we categorize models into D-Models (Deterministic) and E-Models (Exploratory), as illustrated in Figure 2. Despite their

distinct token-level behaviors, both model types are able to achieve partial alignment between P_{result} and P_{task} . Subsequently, Section 2 formalizes and validates this distinction, followed by task-specific comparisons in code generation and recommendation scenarios in Section 3 and an analysis of internal characteristics in Section 4.

The main contributions of this study are as follows: 1) **Probabilistic view of web tasks**: We formulate web tasks as probabilistic sampling processes with a task-level target distribution P_{task} , and evaluate the alignment between P_{token} , P_{task} , and P_{result} . Meanwhile, we identify and validate two model types—D (Deterministic) and E (Exploratory). 2) **Task-specific comparison**: We evaluate both types across task-specific scenarios (e.g., code generation and recommendation), revealing a systematic diversity–stability trade-off and offering insights into when each model type performs favorably. 3) **Internal mechanism analysis**: We further probe the internal mechanisms of both types, verifying their behavioral differences and uncovering the underlying causal factors.

2 Model Types Validation

In this section, we first formalize the definitions of the D- and E-models. We then verify their existence through simulated distribution tasks with explicitly defined P_{task} , comparing the alignment among P_{token} , P_{result} , and P_{task} . Finally, we further validate the two models in a more practical setting.

2.1 Definition of Models

Firstly, considering the discrete nature of the output vocabulary in LLMs, for a discrete random variable $X \in \mathcal{X}$, we define two types of models based on the characteristics of their P_{token} and its alignment with P_{task} . It should be noted that the two types of models do not differ significantly on the P_{result} .

D-model generates a P_{token} that differs significantly from the P_{task} , with one token assigned a substantially higher probability than all others. **E-model** is characterized by a P_{token} that closely aligns with the P_{task} . Formally, for each token generation step t :

$$D\text{-model} \triangleq \left\{ \text{LLM} \mid \forall t, P_{\text{token}}^t(X = x_i) \approx \begin{cases} 1.0, & \text{if } x_i = x_{\text{max}}, \\ 0.0, & \text{otherwise.} \end{cases} \right\},$$

$$E\text{-model} \triangleq \left\{ \text{LLM} \mid \forall t, P_{\text{token}}^t(X = x_i) \approx P_{\text{task}}(x_i), \quad \forall x_i \in \mathcal{X} \right\},$$

where x_{max} refers to the token with the highest probability. D-model results in a highly concentrated distribution with large fluctuations, leading to more diverse and unpredictable outputs. E-model indicates that the model generates tokens in a stable manner, with minimal fluctuations in the token distribution, ensuring consistent outputs aligned with the task.

2.2 Experimental Methods

In this section, we first present a simulation experiment to obtain explicit P_{task} , P_{token} and P_{result} . We then validate the two model types in practical tasks with experiments.

2.2.1 Simulated Distribution Task.

Distribution Definition. Given the discrete nature of the output vocabulary of LLMs, the experimental tasks in this section are

designed for discrete probability distribution sampling. This facilitates the alignment between the P_{task} , the P_{token} and the P_{result} . The specific definitions of these three distributions are as follows:

First, P_{task} refers to the target probability distribution that the task requires the model to sample from. For a discrete random variable $X \in \mathcal{X}$, its distribution is defined as:

$$P_{\text{task}}(X = x_i) = p(x_i), \quad \text{where} \quad \sum_{x_i} p(x_i) = 1. \quad (1)$$

In this experiment, we explicitly specify the task requirements and the target distribution $P_{\text{task}}(X)$ (where $X \in \mathcal{X}$) through the prompt, instructing the model to sample according to this distribution. The prompt template is:

Simulated Tasks Prompt Template:

Given the probability distribution: $P_{\text{task}}(X)$, generate a number of hundreds in $\mathcal{X}$, and give the final list (numbers separated by commas) strictly according to the requirements, without adding any text or process description.

Second, P_{token} refers to the token-level probability distribution output by the LLM at each step during the generation process, based on the current context. Specifically, before generating the t -th token, the model outputs the raw logits $L^t(V)$ (where V is the vocabulary, $\mathcal{X} \subset V$), which is converted into a probability distribution $P^t(V)$ via the Softmax function. The next token is then sampled from this distribution. We extract $P^t(X)$, corresponding to X , from $P^t(V)$ and normalize it to obtain $P_{\text{token}}^t(X)$ for that step:

$$P_{\text{token}}^t(X = x_i) = \frac{P^t(x_i)}{\sum_{x' \in \mathcal{X}} P^t(x')}. \quad (2)$$

Third, P_{result} refers to the empirical probability distribution presented by the number list parsed from the model's final output. Specifically, the sequence of numbers is extracted from the generated string, and the probability is approximated by frequency:

$$P_{\text{result}}(X = x_i) = \frac{\text{count}(x_i)}{N}, \quad (3)$$

where N is the length of the list.

Evaluation Metrics. First, we define **e-score** to quantify the degree of extremeness in the model's P_{token}^t , reflecting the model's tendency to output concentrated distributions. It is computed as the average of the maximum probability value p_{max}^t in the vocabulary probability distribution at each generation step: $\text{e-score} = \frac{1}{T} \sum_{t=1}^T p_{\text{max}}^t$. This metric helps distinguish between D-Models (high e-scores) and E-Models (low e-scores).

Second, the Average Total Variation Distance (ATVD) measures the dispersion between distributions. Specifically, for any two distributions $P_A, P_B \subset P_{\text{task}}, P_{\text{task}}, P_{\text{token}}$, the formal definition of the ATVD between them is as follows:

$$\text{ATVD}(P_A, P_B) = \frac{1}{n} \sum_{i=1}^n |P_A(X = x_i) - P_B(X = x_i)|, \quad (4)$$

where n is the number of possible values of X . Here, $P_{\text{token}} = \frac{1}{T} \sum_{t=1}^T P_{\text{token}}^t$, which represents the overall value.

Additionally, to measure the difference between P_{token} and the P_{task} at each finer-grained step, we further compute ATVD-step:

$$\text{ATVD-step} = \frac{1}{nT} \sum_{t=1}^T \sum_{i=1}^n |P_{\text{token}}^t(X = x_i) - P_{\text{task}}(X = x_i)|. \quad (5)$$

2.2.2 Practical Tasks. To further validate the characteristics of D-model versus E-model, we analyzed the extremity of P_{token} in tasks without explicitly specified probability distributions. Specifically, we conducted experiments on the MMLU benchmark, which comprises multiple categories of single-choice questions. By strictly prompting the LLMs to directly output options, we obtained the probability distribution corresponding to the first token generated by the LLM and recorded the e-score.

2.3 Experimental Setups

2.3.1 Simulated Distribution Tasks. To systematically evaluate the probability distribution sampling capability of LLMs, two types of representative distribution tasks are designed: **Extreme Distribution Task**: $\{1 : 0.1, 2 : 0.7, 3 : 0.1, 4 : 0.1\}$, exhibiting highly concentrated characteristics. **Flat Distribution Task**: $\{1 : 0.1, 2 : 0.1, 3 : 0.1, 4 : 0.2, 5 : 0.1, 6 : 0.1, 7 : 0.1, 8 : 0.1, 9 : 0.1\}$, exhibiting approximately uniform characteristics.

For each model-task combination, we perform 10 independent sampling experiments. The final metrics are calculated as the average of these 10 runs to mitigate the impact of random fluctuations.

2.3.2 Baseline Models. This study selects multiple LLMs as baseline models, covering both open-source and closed-source categories. **Open-source Models**: Llama-3.1-8B-Instruct [8], Qwen-2.5-7B-Instruct [24], DeepSeek-R1-Distill-Qwen-7B [4], Mistral-Small-24B-Instruct-2501¹, Gemma-2-9B-IT [25], DeepSeek-v3 [5]; **Closed-source Models**: GPT-3.5-Turbo, GPT-4o². Among them, the closed-source model and the DeepSeek-v3 are accessed via API.

It should be noted that due to API limitations, the API-accessed models only provide the logits values for the Top-5 tokens. Experimental observations indicate that the sum of the probability mass for these Top-5 tokens is close to 1.0, suggesting that the probabilities of the remaining tokens are negligible. Therefore, for elements in the target subset X that do not appear in the Top-5, their logits values are uniformly set to negative infinity.

2.3.3 Parameter Settings. To obtain raw log probabilities, we set the temperature parameter to 1.0 in these experiments. Unless specified otherwise, all subsequent experiments in this work were conducted at a temperature of 1.0.

2.4 Experimental Results

2.4.1 Sampling Performance. The overall alignment of P_{token} and P_{result} with P_{task} is limited, as shown in Table 1. In the extreme distribution task, in particular, the $\text{ATVD}(P_{\text{task}}, P_{\text{result}})$ and $\text{ATVD}(P_{\text{task}}, P_{\text{token}})$ concentrate near 0.10, implying an average probability gap of about 0.10 per x . This reveals that while current LLMs can sample simple distributions, they struggle to model distributions precisely. Conversely, in flat distribution task, the models' ATVD values are consistently lower, indicating better sampling

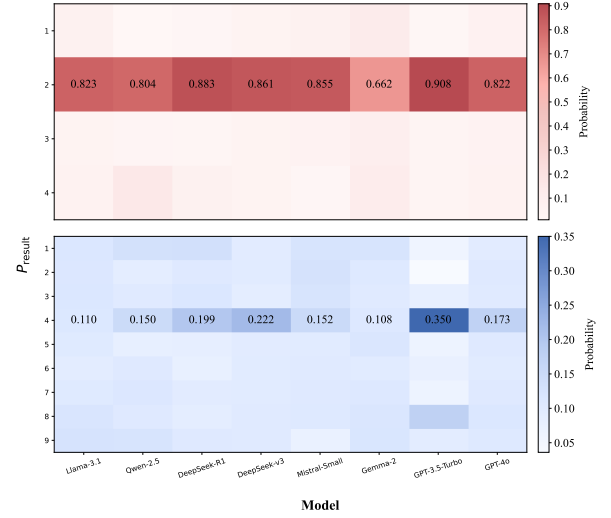


Figure 3: The P_{result} of various models. In the extreme task (top), $P_{\text{result}}(2) > P_{\text{task}}(2) = 0.7$, whereas in the flat task (bottom), $P_{\text{result}}(4) > P_{\text{task}}(4) = 0.2$ for most models.

performance for more uniform distributions. In short, LLMs exhibit greater uncertainty and bias when the target distribution is extreme, but align more reliably when it is nearly uniform.

The model's internal probability distribution closely aligns with its sampling behavior. As shown in Table 1, $\text{ATVD}(P_{\text{token}}, P_{\text{result}})$ remains below 0.04 across all models; for example, Qwen-2.5 reaches 0.005 in the extreme distribution task. This consistency indicates that P_{token} effectively drives P_{result} , reflecting a strong synergistic relationship between them.

Additionally, as shown in Figure 3, we observe a clear monotonic pattern: tokens x with higher $P_{\text{task}}(x)$ tend to receive higher $P_{\text{token}}(x)$ and appear higher $P_{\text{result}}(x)$; conversely, tokens with lower $P_{\text{task}}(x)$ receive lower $P_{\text{token}}(x)$ and $P_{\text{result}}(x)$. We hypothesize that LLMs track relative probabilities via natural language understanding, rather than mapping absolute magnitudes faithfully, leading to systematic overestimation of high-probability items and underestimation of low-probability ones. Moreover, for nearly all models we find $\text{ATVD-step} > \text{ATVD}(P_{\text{task}}, P_{\text{token}})$, highlighting a finer-grained mismatch between P_{token} and P_{task} at the step level.

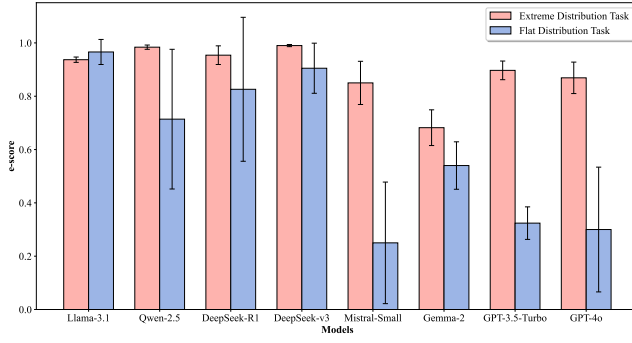
2.4.2 Two Patterns Analysis. In probability distribution sampling tasks, LLMs show two distinct patterns as shown in Figure 4, especially in the flat distribution task. Based on these patterns, we categorize them into two model types as defined in Section 2.1. **D-models** (Llama-3.1, Qwen-2.5, DS-R1-Qwen-2.5, and DeepSeek-v3) consistently exhibit highly concentrated distributions, with e-scores generally above 0.9. This indicates extreme confidence, where the maximum probability approaches 1.0 regardless of task, revealing a tendency toward pattern fixation even when a uniform output is required. **E-models** (Mistral-Small, Gemma-2, GPT-3.5-Turbo, GPT-4o) display greater task-dependent adaptability. In the extreme distribution task, where the maximum probability in P_{task} is 0.7, E-models exhibit relatively high e-scores (e.g., GPT-3.5-Turbo reaches 0.897), though still lower than D-models.

¹<https://mistral.ai/news/mistral-small-3>

²<https://chat.openai.com>

Table 1: Sampling Performance Comparison of Models. Boldface indicates the best results under the corresponding metrics.

Group	Tasks	Extreme distribution				Flat distribution			
	Metrics	ATVD-step ↓	ATVD ↓		Tok/res	ATVD-step ↓	ATVD ↓		Tok/res
	Model	Task/tok	Task/tok	Task/res		Task/tok	Task/tok	Task/res	
D-model	Llama-3.1	0.172	0.080 ± 0.021	0.080 ± 0.022	0.009 ± 0.005	0.193	0.026 ± 0.012	0.028 ± 0.013	0.003 ± 0.005
	Qwen-2.5	0.201	0.158 ± 0.070	0.159 ± 0.070	0.005 ± 0.002	0.151	0.023 ± 0.009	0.025 ± 0.010	0.014 ± 0.008
	DS-R1-Qwen-2.5	0.157	0.128 ± 0.038	0.131 ± 0.041	0.009 ± 0.005	0.165	0.031 ± 0.020	0.034 ± 0.020	0.008 ± 0.009
	Deepseek-v3	0.189	0.080 ± 0.059	0.081 ± 0.061	0.005 ± 0.004	0.182	0.016 ± 0.009	0.011 ± 0.007	0.008 ± 0.005
	AVG	0.179	0.112 ± 0.047	0.113 ± 0.049	0.007 ± 0.004	0.173	0.024 ± 0.013	0.024 ± 0.013	0.008 ± 0.007
E-model	Mistral-Small	0.090	0.084 ± 0.027	0.092 ± 0.031	0.018 ± 0.006	0.048	0.022 ± 0.016	0.036 ± 0.016	0.023 ± 0.010
	Gemma-2	0.090	0.036 ± 0.017	0.042 ± 0.026	0.024 ± 0.014	0.131	0.024 ± 0.004	0.021 ± 0.003	0.016 ± 0.006
	GPT-3.5-Turbo	0.099	0.098 ± 0.018	0.104 ± 0.024	0.015 ± 0.009	0.094	0.082 ± 0.010	0.055 ± 0.023	0.040 ± 0.011
	GPT-4o	0.116	0.082 ± 0.024	0.089 ± 0.026	0.016 ± 0.007	0.106	0.031 ± 0.012	0.013 ± 0.007	0.035 ± 0.016
	AVG	0.101	0.075 ± 0.022	0.082 ± 0.027	0.018 ± 0.009	0.095	0.040 ± 0.011	0.031 ± 0.012	0.029 ± 0.011

**Figure 4: E-scores comparison of models, with error bars representing standard deviations.**

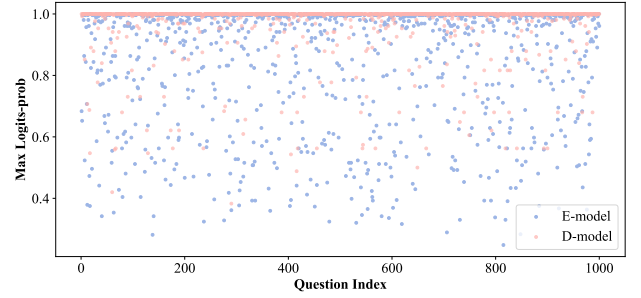
Conversely, in the flat distribution task, where the maximum probability in P_{task} is only 0.2, they show significantly lower e-scores.

Crucially, although E-models exhibit lower e-scores than D-models and show some adaptation to P_{task} , all their e-scores remain higher than the maximum probability in the target P_{task} . This suggests that while E-models can adjust distribution flatness according to the task, they still display a consistent bias, producing probability distributions more concentrated than required and failing to achieve full sampling fidelity.

The two model types also differ across task settings: E-models perform better in extreme-distribution scenarios, whereas D-models excel in flat-distribution ones. As shown in Table 1, in the extreme distribution task D-models yield higher average ATVD($P_{\text{task}}, P_{\text{result}}$) (0.113) than E-models (0.082), while in the flat distribution task D-models reach lower ATVD values (0.024) than E-models (0.031).

Analysis of ATVD-step further indicates that D-models exhibit larger stepwise fluctuations, with generated tokens deviating more from P_{task} (e.g., Qwen-2.5 shows 0.201). In contrast, E-models show ATVD-step values consistent with ATVD(task, token), reflecting higher intra-step stability. Details of the P_{token} evolution during generation are provided in Appendix A.1.

2.4.3 Further Validation of Model Classification. The experimental results are shown in the Figure 5, for e-score results of additional

**Figure 5: The distribution of maximum logits probabilities for 1,000 random problems in MMLU dataset, as demonstrated by D-model Qwen-2.5 and E-model Mistral-Small.**

models, see the Appendix A.3. During generation, the probability mass of the maximum-probability token under the D-model remains concentrated near the top, whereas under the E-model it is more dispersed at comparatively lower values. This demonstrates that, for real-world tasks, the D-model exhibits highly confident token probabilities and a relatively concentrated P_{token} , while the E-model generates more conservative P_{token} based on task content.

3 Task-Specific Properties Comparison

3.1 Experimental Methods

In this experiment, we propose that LLMs implicitly model task-specific probability distributions when processing particular tasks. For a given context, there exists an underlying task-level distribution P_{task} , from which LLMs sample to produce the result distribution $P_{\text{result}} \sim P(\text{result} | \text{context})$. Concretely, the model predicts tokens autoregressively by sampling from the token-level distribution $P_{\text{token}} \sim P(\text{token} | \text{context})$. This process relies on the model’s ability to model the probability distribution at each step of the generation process. We then analyze the performance of two model categories (D- and E-models) across distinct task scenarios to examine their effectiveness in task-specific applications.

3.1.1 Code Generation.

Distribution Definition. The objective of code generation is to infer an appropriate function implementation from a given context. We model this process as a conditional probability distribution, $P_{\text{task}} \sim P(\text{code} \mid \text{problem})$, indicating that, given the problem description, the model seeks to generate code samples with high probability under this distribution.

Evaluation Methodology. To quantify the diversity of model responses, except for the response’s vector cosine similarity, we combine the traditional pass@5 and pass@1 metrics with an exponential scaling that accounts for the increasing difficulty of improvement at higher accuracy levels. We define $\Delta\text{pass} = e^{\text{pass}@5} - e^{\text{pass}@1}$. A larger Δpass indicates that the model explores a broader solution space and generates more diverse candidate answers.

3.1.2 Recommendation.

Distribution Definition. The goal of recommendations is to predict the items most likely to interest a user given their profile and interaction history. We formulate this as a conditional probability distribution, $P_{\text{task}} \sim P(\text{items} \mid \text{profile}, \text{history})$, where the model estimates item probabilities and selects the top- k recommendations.

Candidate items are first obtained using UCF (User-based Collaborative Filtering) and ICF (Item-based Collaborative Filtering) to generate n candidates. The LLM is then prompted to recommend items based on the user information and these candidates, following the prompt template detailed in Appendix C.

Evaluation Metrics. We select candidate items generated by UCF and ICF methods, and task the model with choosing the top- k items for recommendation. We use precision to evaluate the quality of the recommendations’ rankings. The can-hit rate reflects the model’s recommendation divergence by measuring the proportion of predicted recommended items that appear in the candidate list.

3.2 Experimental Setups

Datasets. We use the HumanEval benchmark [2] for the code task. For the recommendation, we use the MovieLens-1M dataset [12], which includes user rating data and item information.

Hyperparameters. In the recommendation task, both UCF and ICF each generate 10 candidates. The LLM generates $k = 10$ recommendations. Temperature settings follow Section 2.3.3.

Models. The baseline models used in this section are the same as those described in Section 2.3.2. In the code generation, we employ Qwen3-Embedding-4B [34] to convert the generated text into embedding vectors and to compute the similarity.

3.3 Experimental Results

It is evident from the experimental results that the D-model and the E-model demonstrate disparate performance in code and recommendation evaluations. This observation is indicative of the presence of discrepancies in their generative capabilities and task adaptability. The D-model exhibits stronger deterministic planning, yet it tends to deviate from target distributions and constraints during generation. Conversely, the E-model maintains better constraint adherence during generation, although it may engender unnecessary diversity overall.

Table 2: Performance for Code and Recommendation Tasks. The DS-R1-Qwen-2.5 lacks Δpass as it rarely produced complete outputs even with a greatly increased max-token limit.

Group	Tasks	Code		Recommendation	
	Model	Δpass	sim	Prec	Can-hit
D-model	Llama-3.1	0.444	0.941	0.172	0.231
	Qwen-2.5	0.368	0.943	0.138	0.175
	DS-R1-Qwen-2.5	-	0.945	0.296	0.421
	Deepseek-v3	0.080	0.981	0.350	0.500
	AVG	0.297	0.952	0.179	0.475
E-model	Mistral-Small	0.272	0.783	0.320	0.466
	Gemma-2	0.209	0.929	0.343	0.482
	GPT-3.5-Turbo	0.351	0.912	0.348	0.496
	GPT-4o	0.183	0.942	0.341	0.499
	AVG	0.264	0.893	0.337	0.491

In the code generation task, as illustrated in the Table 2, the code fragments generated by the D-model through multiple passes demonstrate a higher degree of similarity among themselves and achieve a higher Δpass value when compared to the E-model. This mechanism is rooted in the D-model’s deterministic planning, wherein P_{token} exhibits significant concentration at each step. The decoding process entails the introduction of minor perturbations to the primary code, while preserving the fundamental deterministic content. These minor adjustments enable successive passes to refine details in the solution, leading to higher values for the Δpass . In contrast, the E-model’s flatter P_{token} leads to broader exploration, but in syntax-sensitive code tasks, global changes more easily introduce semantic or execution errors, yielding lower Δpass .

In the recommendation task, as illustrated in the Table 2, the D-model demonstrates a substantially diminished can-hit in comparison to the E-model. This finding signifies a pronounced inclination towards generating a more diverse array of entries beyond the confines of the candidate list, accompanied by a notable deviation from the task’s fundamental requirement of selecting from candidates. Conversely, the E-model demonstrates a higher can-hit value, suggesting that its recommendations are more likely to select items from the candidate list. The outcomes of the E-model have been found to be more concentrated and accurate, demonstrating a more stable alignment with P_{task} .

Overall, for deterministic tasks (e.g., mathematics, code generation), the D-model has been demonstrated to generate deterministic solutions and refine minor errors through multiple iterations. Conversely, the E-model’s task alignment during generation introduces greater uncertainty that may affect performance. However, in tasks involving explicit candidates (e.g., recommendation, search), the D-model’s global planning may deviate confidently from candidates during generation, while the E-model’s alignment during generation yields more accurate results.

4 Internal Characteristics Analysis

In this section, we analyze the internal characteristics of the two model types based on simulation experiments in Section 2.2.1.

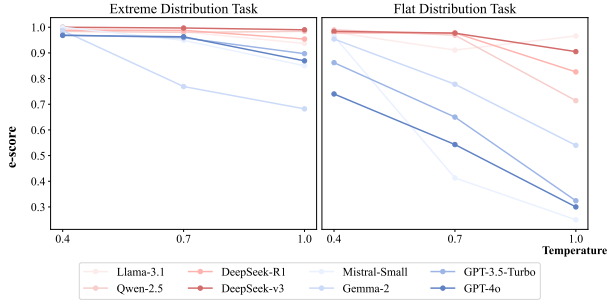


Figure 6: The e-scores of various models change with temperature. Among them, the first four models in the legend are D-models, while the last four are E-models.

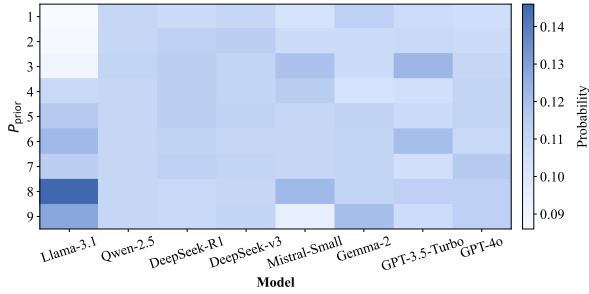


Figure 7: Verify the prior distribution P_{result} for each model. For the same model, the closer the colours of the different numbers are, the more uniform the prior distribution.

4.1 Temperature

By varying the temperature parameter, we examined its effect on the extremity of P_{token} in model outputs. Results indicate that E-models are more temperature-sensitive and exhibit stronger controllability, allowing flexible adaptation across tasks, while D-models tend to maintain low-entropy sampling.

As shown in Figure 6, increasing temperature leads to lower e-scores for all models, reflecting flatter probability distributions and reduced extremity. However, the decline in e-score for D-models is notably smaller than that for E-models.

This pattern aligns with temperature’s role in reshaping the original distribution through scaling. For D-models, even under higher temperatures, P_{token} remains sharply concentrated, suggesting that the model consistently leans toward a high-confidence option in token prediction, and temperature adjustments can hardly significantly alter its extremely confident behavior. In contrast, E-models display greater sensitivity, as their flatter baseline distributions enable more effective temperature-based control. Temperature can be lowered to enhance certainty in deterministic tasks, while it can be raised to promote diverse generation in creative tasks.

4.2 Prior Analysis

To eliminate potential biases arising from intrinsic numeric preferences that could affect the resulting probability distributions, we

Table 3: Prior Validation Across Models

Group	Model	Task/res	ATVD ↓ Task/tok	Tok/res
D-model	Llama-3.1	0.021 ± 0.016	0.024 ± 0.011	0.017 ± 0.008
	Qwen-2.5	0.001 ± 0.000	0.002 ± 0.001	0.001 ± 0.001
	DS-R1-Qwen-2.5	0.005 ± 0.004	0.008 ± 0.004	0.009 ± 0.004
	DeepSeek-v3	0.002 ± 0.000	0.008 ± 0.000	0.008 ± 0.000
	AVG	0.007 ± 0.005	0.011 ± 0.004	0.009 ± 0.003
E-model	Mistral-Small	0.017 ± 0.007	0.012 ± 0.005	0.026 ± 0.004
	Gemma-2	0.005 ± 0.003	0.020 ± 0.005	0.022 ± 0.007
	GPT-3.5-Turbo	0.014 ± 0.013	0.035 ± 0.022	0.042 ± 0.028
	GPT-4o	0.004 ± 0.002	0.016 ± 0.005	0.018 ± 0.005
	AVG	0.010 ± 0.006	0.021 ± 0.009	0.027 ± 0.011

analyze the prior probabilities over the possible values x_i of the discrete random variable X . The results indicate that the tested LLMs exhibit no significant prior bias toward any particular x_i .

Specifically, without specifying any explicit target distribution, we ask the LLMs to randomly generate samples of the target random variable $X \in \mathcal{X} = \{x_i\}$, yielding an empirical prior distribution $\hat{P}_{\text{prior}}$. The prompt template is detailed in the Appendix C.

We then compute the ATVD as in Section 2.2.1. As shown in the Tables 3 and Figure 7, the ATVD values are generally smaller than in the simulated-distribution setting, and $\hat{P}_{\text{prior}}$ is highly similar to the uniform distribution $U(\mathcal{X})$ except for Llama-3.1.

This indicates that, under our setups, the LLM’s prior bias over $\mathcal{X}$ is negligible; therefore, the deviations from P_{task} observed previously are more likely attributable to the sampling mechanism along $P_{\text{token}} \rightarrow P_{\text{result}}$, rather than to any inherent preference for particular numbers.

4.3 Layer Probability Detection

To further analyze the generation mechanisms of D-Models and E-Models, we track the layer-wise evolution of P_{token} across model layers.

Specifically, using an intervention-based activation extraction method, we sampled hidden states from each layer and reconstructed layer-wise logits through the final layer’s normalization and projection mechanisms. By applying the Softmax to target token values, we obtained intermediate-layer distributions P_{token}^l , allowing visualization and quantitative analysis of their evolution.

Due to API limitations, experiments were conducted on open-source models only, except for DeepSeek-v3. Figure 8 shows layer-wise probability changes under extreme and flat tasks. Except for Gemma-2, both D- and E-Models display relatively uniform probabilities in early layers that sharply concentrate near the output, referred to as the *up-layer*, where target semantics are integrated and the dominant token is determined.

Beyond the up-layer, the two model types diverge. In extreme tasks, the D-Model maintains nearly deterministic probabilities (close to 1.0), indicating strong confidence, while the E-Model gradually converges toward the task distribution P_{task} . In flat tasks, the D-Model still deviates from P_{task} , whereas the E-Model aligns P_{token}^l more closely with P_{task} , demonstrating better distribution calibration.

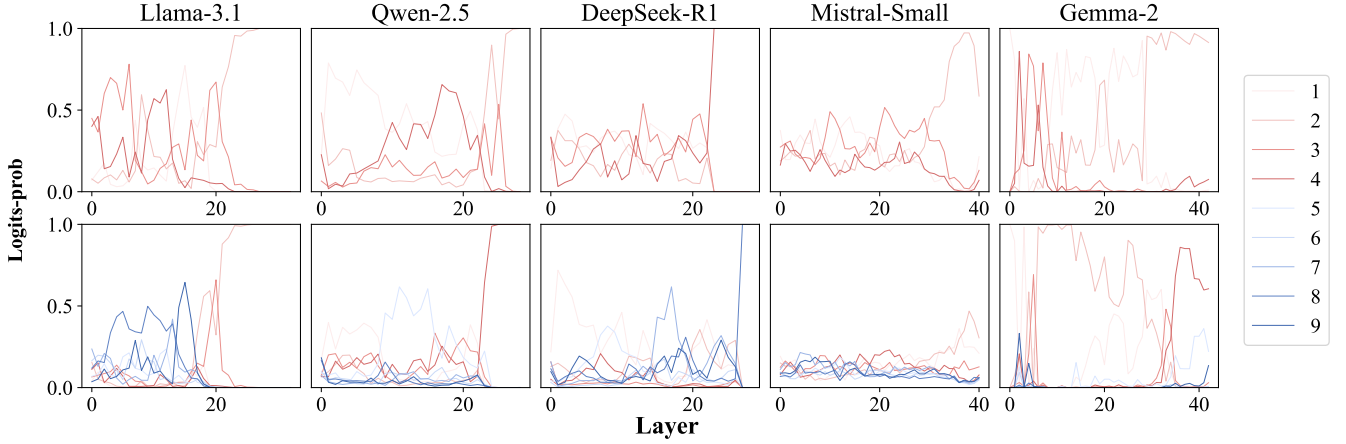


Figure 8: Detection of intermediate-layer probability evolution. Columns correspond to Llama-3.1, Qwen-2.5, DS-R1-Qwen-2.5, Mistral-Small, and Gemma-2. Row 1: Extreme distribution task results; Row 2: Flat distribution task results.

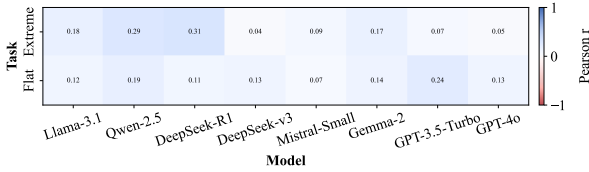


Figure 9: Pearson correlation coefficients between d_i^t and $\Delta P_{\text{token}}^t (X = x_i)$ across models on extreme and flat tasks.

In summary, E-Models demonstrate stronger probability convergence and alignment with P_{task} , enabling dynamic adjustment of token probabilities according to task goals. Conversely, D-Models tend to preserve extreme confidence, leading to overly deterministic outputs and reduced flexibility in tasks requiring diversity.

4.4 Quota Compensation Analysis

In this section, we investigate how, in the D-model, the final sampling distribution can resemble that of the E-model even when P_{token} is extremely peaked. To explain this phenomenon, we hypothesize that the D-model’s sampling follows a quota-compensation mechanism. The findings suggest that this hypothesis is invalid, thereby indicating that the D-model is more likely to adopt a deterministic global planning approach.

At generation step t , let the empirical distribution of the already generated sequence be P_{result}^t . For any token $x_i \in X$, if there is a large discrepancy between $P_{\text{result}}^t (X = x_i)$ and the target distribution $P_{\text{task}} (X = x_i)$, we posit that the model compensates by increasing the logit of x_i when generating the next token, thereby raising $P_{\text{token}}^{t+1} (X = x_i)$ and manifesting the E-model’s extremeness.

We define the residual $d_i^t = P_{\text{task}} (X = x_i) - P_{\text{result}}^t (X = x_i)$. If $|d_i^t|$ is large, i.e., the current token frequency departs substantially from the target, we assume a probability adjustment:

$$\Delta P_{\text{token}}^t (X = x_i) = P_{\text{token}}^{t+1} (X = x_i) - P_{\text{token}}^t (X = x_i) = \lambda \cdot d_i^t,$$

where λ is a compensation coefficient. Under this mechanism, the D-model adjusts logits during generation so that, despite extreme steps, the produced tokens tend to align with P_{task} , potentially yielding a P_{result} similar to E-model.

We test this hypothesis via regression, computing the Pearson correlation coefficient r between d_i^t and $\Delta P_{\text{token}}^t (X = x_i)$. The results (see Figure 9) show very low correlations—even with $r < 0.1$. Hence, the D-model does not follow this quota compensation to align with P_{task} . Instead, it may rely on a global plan as we stated in the definition 2.1, which we leave for future investigation.

5 Conclusion

From a novel probabilistic-sampling perspective, we explore the generation process of LLMs by focusing on the alignment between the token-level probability P_{token} and the task-level distribution P_{task} . Through validation on simulated distribution-sampling tasks and systematic evaluation in downstream settings (code generation and recommendation), we identify and characterize two distributional behavior patterns (D-Models and E-Models), observing consistent differences in the diversity–stability trade-off and in distributional alignment. Further analyses, such as layer-wise probabilities and the quota-compensation hypothesis, provide converging evidence for these two mechanisms. For web scenarios (search, recommendation, and conversational agents), our diagnostics promote transparency of sampling behavior and inform model choice and sampling configuration, enabling a better balance between diversity and reliability under real-world uncertainty.

6 Acknowledgments

This work was supported by the Beijing Nova Program under Grants No. 20250484765, the National Natural Science Foundation of China (NSFC) under Grants No. 62276248, the Key Research and Development Program of Xinjiang Uyghur Autonomous Region Grant No. 2024B03026, the Strategic Priority Research Program of the CAS under Grants No. XDB0680302, and the Youth Innovation Promotion Association CAS under Grants No. 2023111.

References

- [1] Evan Becker and Stefano Soatto. 2024. Cycles of Thought: Measuring LLM Confidence through Stable Explanations. arXiv:2406.03441 doi:10.48550/ARXIV.2406.03441
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. (2021). arXiv:2107.03374 [cs.LG]
- [3] Qi Cheng, Michael Boratko, Pranay Kumar Yelugam, Tim O’Gorman, Nalini Singh, Andrew McCallum, and Xiang Li. 2024. Every Answer Matters: Evaluating Commonsense with Probabilistic Measures. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 493–506. doi:10.18653/v1/2024.acl-long.29
- [4] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- [5] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, and et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [6] Mouly Dewan, Jiqun Liu, Aditya Gautam, and Chirag Shah. 2025. LLM-Driven Usefulness Judgment for Web Search Evaluation. arXiv:2504.14401 [cs.IR] <https://arxiv.org/abs/2504.14401>
- [7] Nate Gillman, Daksh Aggarwal, Michael Freeman, and Chen Sun. 2025. Fourier Head: Helping Large Language Models Learn Complex Probability Distributions. <https://openreview.net/forum?id=4hPwLg7zD3>
- [8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [9] Bowen Gu, Rishi J. Desai, Kueiyu Joshua Lin, and Jie Yang. 2024. Probabilistic medical predictions of large language models. doi:10.1038/S41746-024-01366-4
- [10] Jia Gu, Liang Pang, Huawei Shen, and Xueqi Cheng. 2025. Do LLMs Play Dice? Exploring Probability Distribution Sampling in Large Language Models for Behavioral Simulation. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Henda Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 5375–5390. <https://aclanthology.org/2025.coling-main.360/>
- [11] Ritwik Gupta, Rodolfo Corona, Jiaxin Ge, Eric Wang, Dan Klein, Trevor Darrell, and David M. Chan. 2025. Enough Coin Flips Can Make LLMs Act Bayesian. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 7634–7655. doi:10.18653/v1/2025.acl-long.377
- [12] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015), 19 pages. doi:10.1145/2827872
- [13] Aspen K Hopkins, Alex Renda, and Michael Carbin. 2023. Can LLMs Generate Random Numbers? Evaluating LLM Sampling in Controlled Domains. <https://openreview.net/forum?id=VhhIK9LjVI>
- [14] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How Can We Know When Language Models Know? On the Calibration of Language Models for Question Answering. *Trans. Assoc. Comput. Linguistics* 9 (2021), 962–977. doi:10.1162/TACL_A_00407
- [15] Sam Johnson, Viet Pham, and Thai Le. 2025. Manipulating LLM Web Agents with Indirect Prompt Injection Attack via HTML Accessibility Tree. arXiv:2507.14799 [cs.CR] <https://arxiv.org/abs/2507.14799>
- [16] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislaw Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. 2022. Language Models (Mostly) Know What They Know. arXiv:2207.05221 doi:10.48550/ARXIV.2207.05221
- [17] Katherine Van Koeveering and Jon M. Kleinberg. 2024. How Random is Random? Evaluating the Randomness and Humanness of LLMs’ Coin Flips. arXiv:2406.00092 doi:10.48550/ARXIV.2406.00092
- [18] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching Models to Express Their Uncertainty in Words. <https://openreview.net/forum?id=8s8K2UZGTZ>
- [19] Daniel Maninger, Leon Chemnitz, Amir Molzani Shariifloo, Jannis Bruggen, and Mira Mezini. 2025. Benchmarking LLMs in Web API Integration Tasks. arXiv:2509.20172 [cs.SE] <https://arxiv.org/abs/2509.20172>
- [20] Nicole Meister, Carlos Guestrin, and Tatsunori Hashimoto. 2025. Benchmarking Distributional Alignment of Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 24–49. doi:10.18653/v1/2025.naacl-long.2
- [21] Aliakbar Nafar, Kristen Brent Venable, and Parisa Kordjamshidi. 2024. Probabilistic Reasoning in Generative Large Language Models. arXiv:2402.09614 doi:10.48550/ARXIV.2402.09614
- [22] Batu Ozturkler, Nikolay Malkin, Zhen Wang, and Nebojsa Jojic. 2023. ThinkSum: Probabilistic reasoning over sets using large language models. 1216–1239 pages. doi:10.18653/V1/2023.ACL-LONG.68
- [23] Gwenyth Portillo Wightman, Alexandra Delucia, and Mark Dredze. 2023. Strength in Numbers: Estimating Confidence of Large Language Models by Prompt Agreement. In *Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023)*, Anaelia Ovalle, Kai-Wei Chang, Ninareh Mehrabi, Yada Pruksachatkun, Aram Galystan, Jwala Dhamala, Apurv Verma, Trista Cao, Anoop Kumar, and Rahul Gupta (Eds.). Association for Computational Linguistics, Toronto, Canada, 326–362. doi:10.18653/v1/2023.trustnlp-1.28
- [24] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuyang Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- [25] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, et al. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118 [cs.CL] <https://arxiv.org/abs/2408.00118>
- [26] An Vo, Mohammad Reza Taesiri, Daeyoung Kim, and Anh Totti Nguyen. 2025. B-score: Detecting biases in large language models using response history. arXiv:2505.18545 [cs.LG] <https://arxiv.org/abs/2505.18545>
- [27] Yue Wan, Xiaowei Jia, and Xiang Lorraine Li. 2025. Unveiling Confirmation Bias in Chain-of-Thought Reasoning. 3788–3804 pages. <https://aclanthology.org/2025.findings-acl.195/>
- [28] Han Wang, Alex Whitworth, Pak Ming Cheung, Zhenjie Zhang, and Krishna Kamath. 2025. LLM-based Relevance Assessment for Web-Scale Search Evaluation at Pinterest. arXiv:2509.03764 [cs.IR] <https://arxiv.org/abs/2509.03764>
- [29] Peter West and Christopher Potts. 2025. Base Models Beat Aligned Models at Randomness and Creativity. arXiv:2505.00047 doi:10.48550/ARXIV.2505.00047
- [30] Junda Wu, Yuxin Xiong, Xintong Li, Zhengmian Hu, Tong Yu, Rui Wang, Xiang Chen, Jingbo Shang, and Julian J. McAuley. 2025. CTRLS: Chain-of-Thought Reasoning via Latent State-Transition. arXiv:2507.08182 doi:10.48550/ARXIV.2507.08182
- [31] Tim Z. Xiao, Johannes Zenn, Zhen Liu, Weiyang Liu, Robert Bamler, and Bernhard Schölkopf. 2025. Flipping Against All Odds: Reducing LLM Coin Flip Bias via Verbalized Rejection Sampling. arXiv:2506.09998 doi:10.48550/ARXIV.2506.09998
- [32] Fanghua Ye, Mingming Yang, Jianhui Pang, Longyue Wang, Derek F. Wong, Emine Yilmaz, Shuming Shi, and Zhaopeng Tu. 2024. Benchmarking LLMs via Uncertainty Quantification. http://papers.nips.cc/paper_files/paper/2024/hash/1bdc065d40203a00bd39831153338bb-Abstract-Datasets_and_Benchmarks_Track.html
- [33] Tianyu Yu, Bo Ji, Shouli Wang, Shu Yao, Zefan Wang, Ganqu Cui, Lifan Yuan, Ning Ding, Yuan Yao, Zhiyuan Liu, Maosong Sun, and Tat-Seng Chua. 2025. RLPR: Extrapolating RLVR to General Domains without Verifiers. arXiv:2506.18254 doi:10.48550/ARXIV.2506.18254
- [34] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models.
- [35] Jian-Qiao Zhu and Thomas L. Griffiths. 2025. Incoherent Probability Judgments in Large Language Models. arXiv:2401.16646 [cs.CL] <https://arxiv.org/abs/2401.16646>

A More Results in Model Validation

In this section, we present further experimental results from validation experiments in Section 2.

A.1 Token Distribution Fluctuations

As shown in the Figure 10, the P_{token} for each model at every step reveal that the D-model is more volatile, while the E-model’s P_{token} remains stable and largely aligns with the P_{task} .

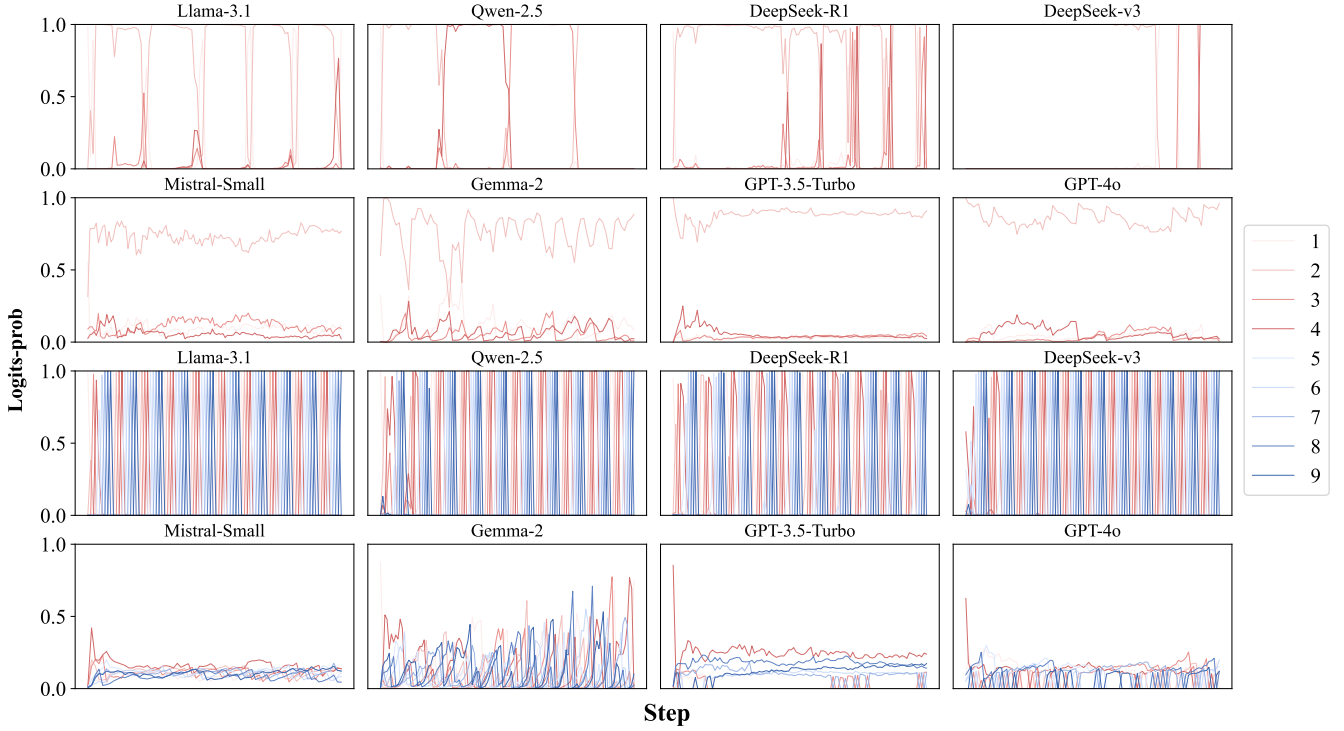


Figure 10: P_{token} at each step for each model. The first two rows correspond to extreme distribution task, while the latter two rows pertain to flat distribution task.

Table 4: Hamming Distance Comparison of D- and E-Models

Group	Model	Distribution tasks	
		Extreme	Flat
D-model	Llama-3.1	0.32	0.36
	Qwen-2.5	0.34	0.82
	DS-R1-Qwen-2.5	0.29	0.72
	DeepSeek-v3	0.19	0.86
	AVG	0.29	0.69
E-model	Mistral-Small	0.33	0.45
	Gemma-2	0.47	0.60
	GPT-3.5-Turbo	0.20	0.79
	GPT-4o	0.33	0.89
	AVG	0.33	0.68

A.2 Diversity Comparison

We evaluate the diversity of model-generated sequences using the Hamming Distance, which measures the number of differing positions between two equal-length sequences. For sequences of unequal length, the shorter one is zero-padded to match the longer.

The analysis shows that sequence differences (mean 0.69 for D-models, 0.68 for E-models) are notably higher in flat distribution tasks than in extreme ones (0.29 for D-models, 0.33 for E-models). This stems from the inherent properties of the distributions: the flat

distribution has a larger value space (9 vs. 4) and a more uniform target probability (maximum 0.2 vs. 0.7), yielding higher sequence randomness. The difference between D-models and E-models remains minor. Notably, Llama-3.1 consistently stays below 0.36, often producing nearly identical or sequential patterns such as “1,2,3,4,5,6,7,8,9...”, indicating limited randomness (see Appendix D).

A.3 Practical Task Results

In this section, we present the e-score results for different models on subsets of the MMLU dataset, including high school mathematics, global facts, and all. The DS-R1-Qwen-2.5 model was excluded because its generation always starts with the special token <think>. As shown in Table 5, the D-model yields higher e-scores than the E-model across most tasks, confirming the coexistence of both types in practice. Notably, Llama-3.1 shows a lower e-score compared with our simulated experiments, which we hypothesize results from this evaluation considering only the first generated token. Llama-3.1 may produce a flatter P_{token} distribution at the initial step, with extreme patterns emerging in subsequent generations.

B Other Experimental Details

B.1 Hardware Setup

We did not employ any quantization methods in this work, nor did we rely on any specific inference frameworks. All model inference was run directly in a local standard environment, with 8 NVIDIA A100 GPUs and Intel Xeon Platinum 8358 CPUs (2.60 GHz).

Table 6: Sampling Examples of Extreme Distribution Task

Model	Extreme Distribution Examples
Llama-3.1	1, 2, 3, 3, 4, 1, 1, 1, 1, 1, 2, ... 3, 3, 3, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ...
Qwen-2.5	1, 2, ...
DS-R1-Qwen-2.5	1, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, ...
DeepSeek-v3	2, 1, 2, 2, 2, 2, 2, 2, 2, ...
Mistral-Small	2, 2, 2, 2, 1, 3, 2, 2, 2, 2, 3, 1, 2, 4, 2, 3, 2, 2, 2, 3, 2, 2, 2, 2, 2, 2, 3, 3, 1, 2, 2, 2, 2, 1, 2, 1, 4, 2, 2, 2, 2, 2, 2, 1, 2, 4, 1, 4, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 1, 3, 2, 3, 2, 2, 4, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, ...
Gemma-2	2, 2, 2, 2, 4, 2, 2, 3, 1, 4, 2, 2, 3, 2, 2, 2, 2, 4, 2, 1, 2, 3, 2, 3, 1, 2, 2, 4, 2, 2, 2, 2, 1, 2, 2, 3, 2, 2, 2, 2, 4, 1, 2, 2, 2, 2, 2, 2, 2, 3, 4, 2, 2, 2, 2, 2, 2, 3, 2, 1, 2, 4, 2, 2, 2, 1, 2, 2, 2, 3, 2, 2, 2, 4, 2, 2, 2, 2, ...
GPT-3.5-Turbo	2, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 4, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 2, 2, 2, 2, 2, 3, ...
GPT-4o	2, 1, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 4, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ...

Table 5: Comparison of E-scores Across Models. HSM and GF denote high school mathematics and global facts.

Group	Model	HSM	GF	all
D-model	Llama-3.1	0.561	0.679	0.734
	Qwen-2.5	0.908	0.903	0.969
	DS-R1-Qwen-2.5	-	-	-
	DeepSeek-v3	0.955	0.973	0.983
	AVG	0.807	0.852	0.895
E-model	Mistral-Small	0.543	0.586	0.845
	Gemma-2	0.815	0.859	0.906
	GPT-3.5-Turbo	0.668	0.683	0.869
	GPT-4o	0.888	0.862	0.927
	AVG	0.728	0.747	0.887

B.2 Detailed Metrics

Pass@k is a metric for evaluating the programming capabilities of generative models. Given the same problem, the model generates

k candidate solutions; if at least one candidate solution passes the preset test cases, the problem is deemed a success. Pass@k measures the probability that the model generates at least one correct solution in multiple attempts, and it is typically used to evaluate the overall performance of code generation models in solving programming tasks. In this paper, we adopt the gap between Pass@5 and Pass@1 to measure the diversity of the code generated by the model.

C Prompt Template

The prompt template of recommendation is:

Recommendation Prompt Template:

You are a professional movie recommendation specialist, please complete the task based on the following information:

user profile: [user profile]

Here are some of the movies he/she has watched and their ratings. You are encouraged to learn his/her movie preference from them: [history]

Here's a list of candidate movies that he/she is likely to like:
[candidates]

Please analyze the user's preferences based on his/her movie viewing history and select the top 10 movies that are most likely to be liked from the list given above.

Be sure to select from the candidate list above and do not generate other movies to ensure that the ids are correct.

Please output only ten movie ids and titles according to the format below without any description.

Output format: "The ten recommended movies are:

n4: Waiting to Exhale (1995), 8: Tom and Huck (1995), 21: Get Shorty (1995), 24: Powder (1995)..."

The prompt template of prior analysis is:

Prior Prompt Template:

Must generate a hundred numbers in X randomly, and give the final list (numbers separated by commas) strictly according to the requirements, without adding any text or process description.

D Sample Examples

This section shows examples of sampling results for different models, as shown in Table 6 and Table 7.

E Related Works

E.1 Log Probabilities in LLMs

The log probabilities output by LLMs have been widely utilized in model interpretability research. On one hand, [3] and [9] respectively validate the effectiveness of using explicit probability outputs for commonsense question answering and medical predictions. However, the latter reveals that most LLMs exhibit an excessively polarized explicit probability distribution, with an overconfidence in predictions that is independent of correctness, suggesting the presence of biases in the model’s intrinsic probability distribution.

Table 7: Sampling Examples of Flat Distribution Task

Model	Flat Distribution Example
Llama-3.1	1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 7, 7, 7, 7, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 9
Qwen-2.5	1, 4, 3, 2, 9, 6, 1, 1, 9, 5, 7, 2, 4, 8, 9, 3, 2, 7, 8, 4, 3, 6, 1, 8, 7, 1, 5, 7, 2, 3, 1, 4, 6, 8, 9, 6, 1, 9, 7, 2, 5, 8, 7, 4, 3, 9, 4, 7, 1, 2, 9, 6, 1, 7, 2, 4, 3, 5, 8, 9, 9, 7, 5, 1, 6, 1, 9, 9, 3, 6, 4, 1, 2, 7, 4, 8, 5, ...
DS-R1-Qwen-2.5	1, 2, 3, 4, 4, 5, 6, 7, 8, 9, 1, 1, 2, 3, 4, 4, 5, 6, 7, 8, 9, 1, 2, 3, 4, 4, 5, 6, 7, 8, 9, 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 3, 4, 5, 6, ...
DeepSeek-v3	4, 5, 9, 1, 2, 4, 7, 8, 4, 6, 3, 4, 9, 2, 7, 1, 4, 5, 6, 8, 3, 4, 9, 1, 2, 4, 7, 8, 4, 6, 3, 4, 9, 2, 7, 1, 4, 5, 6, 8, 3, 4, 9, 1, 2, 4, 7, 8, 4, 6, 3, 4, 9, 2, 7, 1, 4, 5, 6, 8, 3, 4, 9, 1, 2, 4, 7, 8, 4, 6, 3, 4, 9, 2, 7, 1, 4, 5, ...
Mistral-Small	1, 6, 2, 8, 3, 8, 3, 1, 3, 5, 5, 7, 9, 8, 8, 2, 4, 9, 1, 3, 6, 5, 7, 1, 5, 6, 2, 4, 3, 1, 6, 9, 1, 9, 4, 3, 6, 8, 1, 5, 9, 7, 2, 3, 8, 2, 3, 7, 8, 4, 5, 5, 2, 3, 6, 2, 4, 6, 7, 8, 3, 8, 4, 1, 6, 5, 8, 8, 2
Gemma-2	1, 4, 2, 5, 9, 3, 7, 5, 1, 4, 8, 1, 6, 9, 2, 8, 7, 1, 4, 6, 3, 5, 9, 2, 4, 8, 7, 5, 3, 1, 9, 6, 8, 4, 7, 1, 3, 2, 6, 5, 9, 8, 2, 7, 1, 5, 4, 9, 4, 8, 6, 3, 5, 2, 7, 1, 3, 9, 5, 8, 4, 6, 2, 1, 9, 7, 3, 2, 8, 6, 5, 4, 3, 7, 1, 9, 8, 5, ...
GPT-3.5-Turbo	4, 4, 3, 4, 6, 9, 8, 6, 5, 4, 8, 8, 4, 4, 6, 6, 5, 8, 3, 4, 4, 6, 4, 8, 8, 4, 4, 9, 4, 4, 4, 8, 4, 4, 8, 4, 1, 4, 5, 4, 8, 4, 8, 4, 4, 2, 4, 4, 8, 8, 4, 8, 5, 6, 4, 4, 8, 6, 3, 8, 8, 6, 4, 5, 4, 4, 4, 6, 9, 3, 8, 4, 4, 4, 4, 4, ...
GPT-4o	4, 7, 9, 3, 6, 2, 8, 4, 4, 3, 8, 5, 6, 6, 9, 2, 8, 4, 4, 9, 1, 3, 6, 9, 4, 1, 2, 7, 4, 4, 1, 7, 5, 3, 8, 8, 2, 8, 2, 3, 2, 9, 8, 7, 6, 8, 6, 5, 4, 6, 5, 1, 9, 4, 5, 2, 5, 7, 4, 5, 9, 4, 1, 4, 8, 4, 1, 9, 6, 4, 3, 7, 5, 7, 5, 3, ...

On the other hand, numerous studies focus on leveraging LLMs' log probabilities to quantify model uncertainty. [32] proposes a benchmark for evaluating LLM uncertainty based on logits probabilities. [23] demonstrates that combining log probabilities across multiple prompts yields more accurate confidence estimates, while [1] maps model uncertainty through the stability of explanation distributions generated by the model. Calibration-based concepts can assess LLM uncertainty; [18] compares the calibration effects of verbalized probabilities versus probabilities extracted from logits, and [14] confirms that model probabilities significantly deviate from actual accuracy rates in traditional QA tasks. [33] innovatively transforms token probabilities into reinforcement learning

reward signals, enhancing the reasoning capabilities of large models and expanding the application domain of Reinforcement Learning with Verifiable Rewards (RLVR). Notably, [27] approximates the model's internal beliefs through direct probability questioning, empirically demonstrating the presence of confirmation bias in chain-of-thought reasoning.

Overall, prior work utilizes log probabilities but seldom connects token-level (P_{token}) and task-level (P_{task}) analyses, often assuming overconfident outputs while neglecting flat distribution behaviors.

E.2 Probabilistic Reasoning in LLMs

Current research investigates the probabilistic reasoning abilities of LLMs from various perspectives. In probabilistic reasoning, [22] introduces a two-stage probabilistic reasoning framework based on structured object sets, aggregating query probabilities across phrase groups to improve complex reasoning. [30] models reasoning as an explicit probability distribution in latent space, enabling systematic exploration of alternative reasoning paths. [21] evaluates LLMs' capability to perform probabilistic reasoning on text with quantified uncertainty using a Bayesian modeling approach.

However, a significant disconnect exists between LLMs' probabilistic expressiveness and their true sampling capabilities: while [16] verifies that models can quantify uncertainty through confidence levels and that predictive distributions align with real-world frequencies, empirical studies reveal severe limitations. [26] identifies biases in LLMs' ability to generate random numbers; [10], [13], and [31] further point out models' difficulty in faithfully sampling target distributions, yet fail to deeply analyze these defects in conjunction with model generation principles. [17] confirms that LLMs not only struggle to generate unbiased samples but may even amplify human cognitive biases.

This "knowledge-action" discrepancy also manifests in complex tasks: [20] discovers deficiencies when LLMs align with group opinion distributions, with the core contradiction lying in models' ability to describe distributions yet inability to sample from them. Moreover, evaluation metrics based on log probabilities may underestimate the performance gap. More fundamentally, [35] detects incoherence in LLMs' probability judgments through probability identity tests. Recent work attempts to bridge this gap: [11] verifies that, given sufficient contextual examples, LLMs roughly update their estimates of coin bias according to Bayesian rules, while [7] proposes a continuous prior learning method from a probability density estimation perspective, attempting to extend models' ability to model complex distributions. Nonetheless, existing research primarily focuses on comparative analysis between LLMs' generated results and tasks, still lacking systematic analysis of whether the intrinsic token-level distribution P_{token} can adapt to the task-level distribution P_{task} , and failing to clarify the specific impact mechanisms of sampling capability defects in real-world tasks.